

Přednáška 12 OneToMany

Z webu Symfony

Zdroj: <https://symfony.com/doc/6.4/doctrine/associations.html>

<https://www.doctrine-project.org/projects/doctrine-orm/en/2.17/reference/association-mapping.html>

Obsah

1	Jak pracovat s doktrínovými asociacemi / vztahy.....	1
1.1	Sdružení ManyToOne / OneToMany.....	2
1.2	Mapování vztahu ManyToOne.....	3
1.3	Co je ArrayCollection Stuff?.....	5
1.4	Ukládání souvisejících subjektů.....	5
1.5	Aktualizace vztahu z inverzní strany.....	6
1.6	Načítání souvisejících objektů.....	6
1.7	Vztahy a proxy třídy.....	7
1.8	Připojení k souvisejícím záznamům.....	7
1.9	Nastavení informací z inverzní strany.....	8
1.10	Kód z praktického příkladu.....	10
1.10.1	Výpisy souborů.....	10
2	Profiler.....	14
2.1	Pokud po spuštění projektu nejde profiler.....	15
2.2	Vypnutí profileru.....	15
3	Roletka ve formuláři.....	16
4	Form -Dropdownlist.....	16
4.1	Vyplnění dropdownlist formuláře z entity.....	16
4.2	Použití vlastního dotazu pro entity.....	22
5	Generované PDF z HTML v Symfony 6.....	27
6	Data z html formuláře do routy (get).....	30

1 Co je asociace

asociace týká vztahů mezi entitami

Typy asociací

1. **One-to-One (1:1)**: Jeden záznam v jedné tabulce odpovídá jednomu záznamu v jiné tabulce.
2. **One-to-Many (1:N)**: Jeden záznam v jedné tabulce odpovídá více záznamům v jiné tabulce.
3. **Many-to-Many (N:N)**: Více záznamů v jedné tabulce odpovídá více záznamům v jiné tabulce.

2 Praktické vytvoření

Nastavíme databázi v souboru.env

```
DATABASE_URL="mysql://pokus:pokus@127.0.0.1:3306/pokus?serverVersion=mariadb-10.4.28"
```

Vytvoření entity Product

symfony console make:entity Product nebo **php bin/console make:entity Product**

sloupce name: string 255

quantity: string 255

Vytvoření entity Category

symfony console make:entity Product nebo **php bin/console make:entity Category**

2.1 Mapování vztahu ManyToOne

V tomto příkladu může být každá Category spojena s *mnoha* Products. Každý produkt však lze přiřadit pouze k *jedné* kategorii.

Chcete-li to namapovat, nejprve vytvořte vlastnost category na Product třídě s ManyToOneatributem. Můžete to udělat ručně nebo pomocí make:entity příkazu, který vám položí několik otázek o vašem vztahu.

Příkaz

php bin/console make:entity

Class name of the entity to create or update (e.g. BraveChef):

> Product

New property name (press <return> to stop adding fields):

> category

Field type (enter ? to see all types) [string]:

> relation

What class should this entity be related to?:

> Category

Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]:

> ManyToOne

Is the Product.category property allowed to be null (nullable)? (yes/no) [yes]:

> no

Do you want to add a new property to Category so that you can access/update Product objects from it - e.g. \$category->getProducts()? (yes/no) [yes]:

> yes

New field name inside Category [products]:

> products

Do you want to automatically delete orphaned App\Entity\Product objects (orphanRemoval)? (yes/no) [no]:

> no

New property name (press <return> to stop adding fields):

>

(press enter again to finish)

Tím došlo ke změně ve dvou entitách. Nejprve přidal category do Product entity novou vlastnost (a metody getter & setter):

2.2 Výsledek generování

Entita Product.php

```
?php

namespace App\Entity;

use App\Repository\ProductRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: ProductRepository::class)]
class Product
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $name = null;

    #[ORM\Column(length: 255)]
    private ?string $quantity = null;

    #[ORM\ManyToOne(inversedBy: 'products')]
    #[ORM\JoinColumn(nullable: false)]
    private ?Category $category = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getName(): ?string
    {
        return $this->name;
    }

    public function setName(string $name): static
    {

```

```

        $this->name = $name;

        return $this;
    }

    public function getQuantity(): ?string
    {
        return $this->quantity;
    }

    public function setQuantity(string $quantity): static
    {
        $this->quantity = $quantity;

        return $this;
    }

    public function getCategory(): ?Category
    {
        return $this->category;
    }

    public function setCategory(?Category $category): static
    {
        $this->category = $category;

        return $this;
    }
}

```

Entita Category.php

```

<?php

namespace App\Entity;

use App\Repository\CategoryRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

```

```

#[ORM\Entity(repositoryClass: CategoryRepository::class)]
class Category
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $name = null;

    /**
     * @var Collection<int, Product>
     */
    #[ORM\OneToMany(targetEntity: Product::class, mappedBy: 'category')]
    private Collection $products;

    public function __construct()
    {
        $this->products = new ArrayCollection();
    }

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getName(): ?string
    {
        return $this->name;
    }

    public function setName(string $name): static
    {
        $this->name = $name;

        return $this;
    }

    /**
     * @return Collection<int, Product>
     */
    public function getProducts(): Collection

```

```

{
    return $this->products;
}

public function addProduct(Product $product): static
{
    if (!$this->products->contains($product)) {
        $this->products->add($product);
        $product->setCategory($this);
    }

    return $this;
}

public function removeProduct(Product $product): static
{
    if ($this->products->removeElement($product)) {
        // set the owning side to null (unless already changed)
        if ($product->getCategory() === $this) {
            $product->setCategory(null);
        }
    }

    return $this;
}
}

```

Mapování ManyToOne uvedené výše je **povinné**

OneToMany je to **volitelné**: použijte se tehdy tehdy, chcete-li mít přístup k produktům, které souvisejí s kategorií (toto je jedna z otázek, make:entity, které vám položí).

V tomto příkladu bude vhodné volat \$category->getProducts(). Pokud to nechcete, pak také nepotřebujete konfiguraci inversedBy nebo mappedBy .

Vysvětlení

Kód uvnitř __construct() je důležitý: \$productsVlastnost musí být objekt kolekce, který implementuje rozhraní Doctrine Collection. V tomto případě se použije objekt ArrayCollection . Toto vypadá a funguje téměř *přesně* jako pole, ale má určitou přidanou flexibilitu. Jen si představte, že je to arraya budete v dobré kondici.

Vysvětlení 2

```
public function __construct()
{
    $this->products = new ArrayCollection();
}
```

Tato konstruktorová metoda slouží k inicializaci vlastnosti `$products` jako nové instance `ArrayCollection` při vytváření objektu třídy. To znamená, že kdykoli vytvoříte nový objekt této třídy, vlastnost `$products` bude automaticky nastavena na prázdnou kolekci, připravenou k použití.

```
use Doctrine\Common\Collections\ArrayCollection;
```

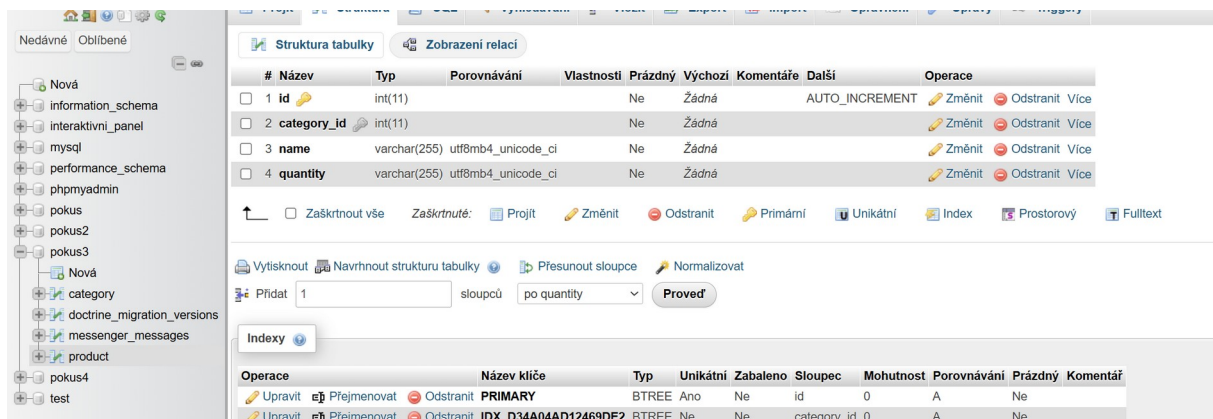
Tato třída je užitečná pro práci s kolekcemi dat v Doctrine ORM a může být použita v jakémkoli PHP projektu, kde je potřeba pokročilá manipulace s poli

Vaše databáze je nastavena! Nyní spusťte migraci jako obvykle:

```
$ php bin/console doctrine:migrations:diff
$ php bin/console doctrine:migrations:migrate
```

Díky vztahu se tak `category_id`na tabulce vytvoří sloupec cizího klíče `product`. Doktrína je připravena přetrvat náš vztah!

V databázi se vytvoří tabulky `Product` a `Category` a také relace



#	Název	Typ	Porovnávání	Vlastnosti	Prázdný	Výchozí	Komentáře	Další	Operace
1	id	int(11)			Ne	Žádná		AUTO_INCREMENT	Změnit Odstranit Více
2	category_id	int(11)			Ne	Žádná			Změnit Odstranit Více
3	name	varchar(255) utf8mb4_unicode_ci			Ne	Žádná			Změnit Odstranit Více
4	quantity	varchar(255) utf8mb4_unicode_ci			Ne	Žádná			Změnit Odstranit Více

Operace	Název klíče	Typ	Unikátní	Zabaleno	Sloupec	Mohutnost	Porovnávání	Prázdný	Komentář
Upravit Přejmenovat Odstranit	PRIMARY	BTREE	Ano	Ne	id	0	A	Ne	
Upravit Přejmenovat Odstranit	IDX_D34A04AD12469DE2	BTREE	Ne	Ne	category_id	0	A	Ne	


```

public function index(EntityManagerInterface $entityManager): Response
{
    $category = new Category();
    $category->setName('Computer Peripherals');

    $product = new Product();
    $product->setName('Keyboard');
    $product->setPrice(19.99);
    $product->setDescription('Ergonomic and stylish!');

    // relates this product to the category
    $product->setCategory($category);

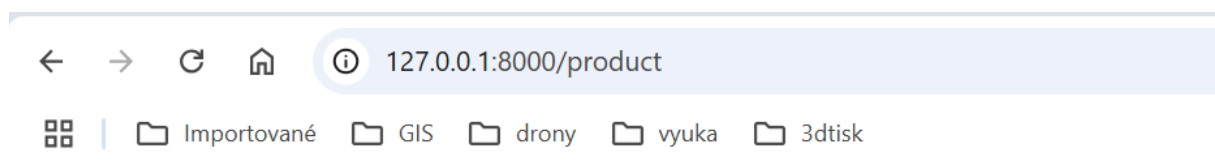
    $entityManager->persist($category);
    $entityManager->persist($product);
    $entityManager->flush();

    return new Response(
        'Saved new product with id: '.$product->getId()
        .' and new category with id: '.$category->getId()
    );
}

```

Když přejdete na /product, přidá se jeden řádek k tabulkám category a product. Sloupec product.category_id pro nový produkt je nastaven na jakoukoli hodnotu id nové kategorie. Doktrína za vás řídí trvání tohoto vztahu:

Pokud jste v ORM nováčkem, je to nejtěžší *koncept* : musíte přestat přemýšlet o své databázi a místo toho přemýšlet *pouze* o svých objektech. Namísto nastavení id celého čísla kategorie na Product, nastavíte celý Category *objekt* . Doctrine se postará o zbytek při ukládání.



Saved new product with id: 1 and new category with id: 1

↔ T ↔

id name

☐

Upravit

Kopírovat

Odstranit

1 Computer Peripherals

↔ T ↔

id category_id name quantity

☐

Upravit

Kopírovat

Odstranit

1 1 Keyboard 10

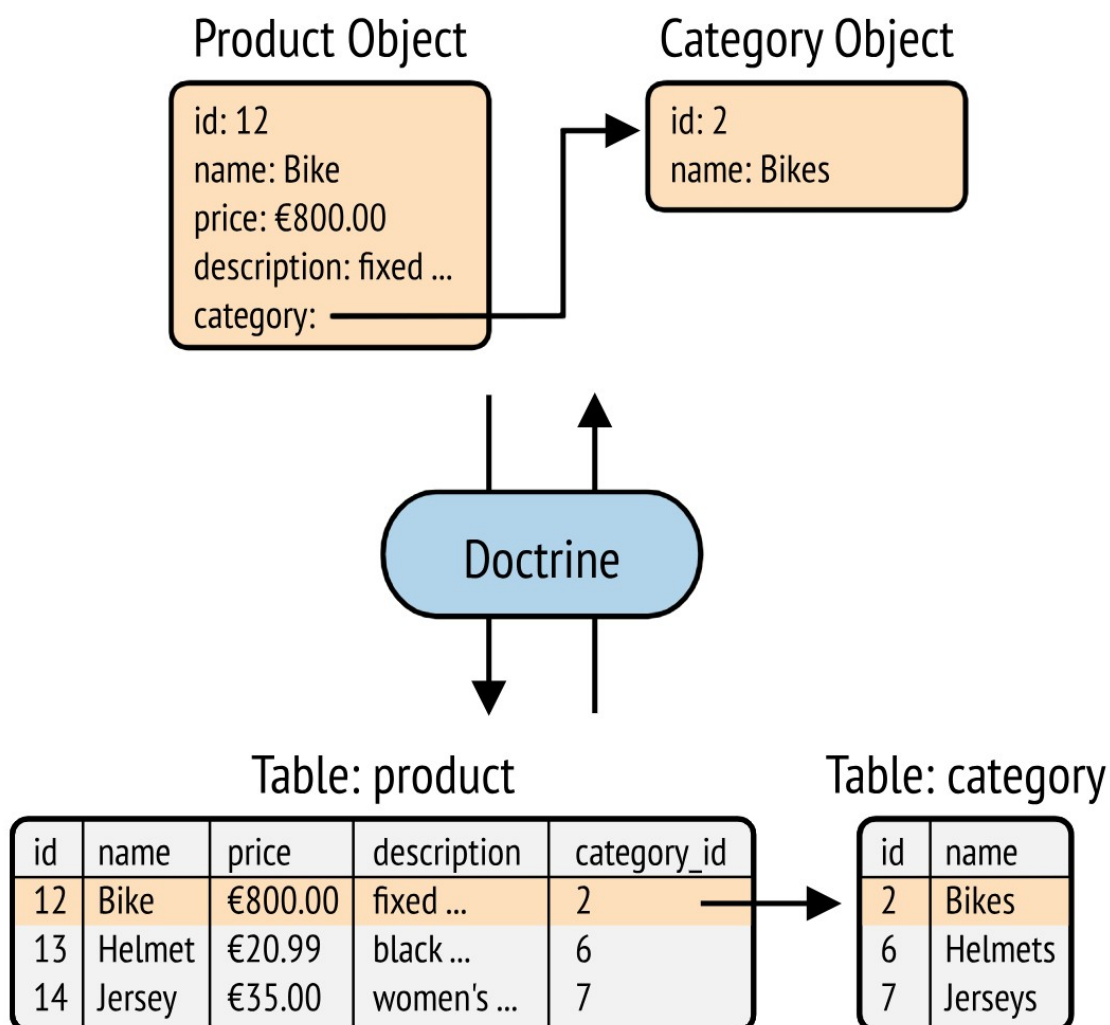
☐ Zaškrtnout vše
 Zaškrtnuté:

Upravit

Kopírovat

Odstranit

Export



2.4 Aktualizace vztahu z inverzní strany

Mohl byste také zavolat `$category->addProduct()`, abyste změnili vztah? Ano, ale jen proto, že `make:entity` nám ten příkaz pomohl. Další podrobnosti naleznete v části: [Associations-inverse-side](#) .

2.5 Načítání souvisejících objektů

Když potřebujete načíst související objekty, váš pracovní postup vypadá jako předtím. Nejprve načtete `$product` objekt a poté získáte přístup k jeho souvisejícímu `Category` objektu:

```
// src/Controller/ProductController.php
namespace App\Controller;

use App\Entity\Product;
// ...

class ProductController extends AbstractController
{
    public function show(ProductRepository $productRepository, int $id):
    Response
    {
        $product = $productRepository->find($id);
        // ...

        $categoryName = $product->getCategory()->getName();

        // ...
    }
}
```

V tomto příkladu nejprve zadáte dotaz na `Product` objekt založený na produktu `id`. To vydá dotaz k načtení *pouze* produktových dat a hydratuje soubor `$product`. Později, když zavoláte `$product->getCategory()->getName()`, Doctrine tiše provede druhý dotaz, aby našel, co `Category` souvisí s tímto `Product`. Připraví `$category` předmět a vrátí vám ho.

Důležitá je skutečnost, že máte přístup ke kategorii související s produktem, ale data kategorie se ve skutečnosti nenačítají, dokud se na kategorii neptáte (tj. je „líně načtena“).

Protože jsme mapovali volitelnou `OneToMany` stranu, můžete se také dotazovat opačným směrem:

```
// src/Controller/ProductController.php
```

```
// ...
class ProductController extends AbstractController
{
    public function showProducts(CategoryRepository $categoryRepository, int
$id): Response
    {
        $category = $categoryRepository->find($id);

        $products = $category->getProducts();

        // ...
    }
}
```

V tomto případě nastanou stejné věci: nejprve se dotážete na jeden Category objekt. Poté, pouze když (a pokud) přistoupíte k produktům, Doctrine provede druhý dotaz k načtení souvisejících Productobjektů. Tomuto dodatečnému dotazu se lze vyhnout přidáním JOINů.

2.6 Vztahy a proxy třídy

Toto "líné načítání" je možné, protože v případě potřeby Doctrine vrátí "proxy" objekt místo skutečného objektu. Podívejte se znovu na výše uvedený příklad:

```
$product = $productRepository->find($id);
```

```
$category = $product->getCategory();
```

```
// prints "Proxies\AppEntityCategoryProxy"
dump(get_class($category));
die();
```

Tento proxy objekt rozšiřuje skutečný Categoryobjekt a vypadá a chová se přesně jako on. Rozdíl je v tom, že pomocí proxy objektu může Doctrine zpozdít dotazování na skutečná Categorydata, dokud tato data skutečně nepotřebujete (např. dokud nezavoláte \$category->getName()).

Třídy proxy jsou generovány Doctrine a uloženy v adresáři mezipaměti. Pravděpodobně si nikdy ani nevšimnete, že váš \$categoryobjekt je ve skutečnosti proxy objektem.

V další části, když načtete data o produktu a kategorii najednou (prostřednictvím spojení), Doctrine vrátí *skutečný* Category objekt, protože není třeba nic líně načítat.

2.7 Připojení k souvisejícím záznamům

Ve výše uvedených příkladech byly provedeny dva dotazy – jeden na původní objekt (např. a Category) a jeden na související objekt(y) (např. Product objekty).

Nezapomeňte, že všechny dotazy provedené během požadavku můžete zobrazit prostřednictvím panelu nástrojů pro ladění webu.

Pokud dopředu víte, že budete potřebovat přístup k oběma objektům, můžete se vyhnout druhému dotazu tím, že v původním dotazu zadáte spojení. Přidejte do třídy následující metodu ProductRepository:

```
// src/Repository/ProductRepository.php

// ...
class ProductRepository extends ServiceEntityRepository
{
    public function findOneByIdJoinedToCategory(int $productId): ?Product
    {
        $entityManager = $this->getEntityManager();

        $query = $entityManager->createQuery(
            'SELECT p, c
            FROM App\Entity\Product p
            INNER JOIN p.category c
            WHERE p.id = :id'
        )->setParameter('id', $productId);

        return $query->getOneOrNullResult();
    }
}
```

To bude *stále* vracet pole Product objektů. Ale nyní, když zavoláte \$product->getCategory() a použijete tato data, nebude proveden žádný druhý dotaz.

Nyní můžete tuto metodu použít ve vašem řadiči k dotazu na Product objekt a související objekt Category v jednom dotazu:

```
// src/Controller/ProductController.php

// ...
class ProductController extends AbstractController
{
    public function show(ProductRepository $productRepository, int $id):
    Response
    {
        $product = $productRepository->findOneByIdJoinedToCategory($id);
    }
}
```

```

        $category = $product->getCategory();

        // ...
    }
}

```

2.8 Nastavení informací z inverzní strany

Doposud jste vztah aktualizovali voláním `$product->setCategory($category)`. To není náhoda! Každý vztah má dvě strany: v tomto příkladu `Product.category` je to *vlastnická* strana a `Category.products` je to *inverzní* strana.

Chcete-li aktualizovat vztah v databázi, *musíte* nastavit vztah na straně *vlastníka*. Vlastnická strana je vždy tam, kde `ManyToOne` je nastaveno mapování (u `ManyToMany` vztahu si můžete vybrat, která strana je vlastníkem).

Znamená to, že není možné zavolat `$category->addProduct()` nebo `$category->removeProduct()` aktualizovat databázi? Ve skutečnosti je možné, díky nějakému chytrému kódu, který `make:entity` příkaz vygeneroval:

```

// src/Entity/Category.php

// ...
class Category
{
    // ...

    public function addProduct(Product $product): self
    {
        if (!$this->products->contains($product)) {
            $this->products[] = $product;
            $product->setCategory($this);
        }

        return $this;
    }
}

```

Klíč je `$product->setCategory($this)` který nastavuje stranu *vlastníka*. Díky tomu se při uložení vztah v databázi aktualizuje.

Co takhle *odstranit* a `Product` z `Category`? Příkaz `make:entity` také vygeneroval `removeProduct()` metodu:

```

// src/Entity/Category.php

```

```

namespace App\Entity;

// ...
class Category
{
    // ...

    public function removeProduct(Product $product): self
    {
        if ($this->products->contains($product)) {
            $this->products->removeElement($product);
            // set the owning side to null (unless already changed)
            if ($product->getCategory() === $this) {
                $product->setCategory(null);
            }
        }

        return $this;
    }
}

```

Díky tomu, pokud zavoláte `$category->removeProduct($product)`, bude v databázi nastaveno `category_id` na `.Productnull`

Ale místo nastavení `category_id` na `null`, co když chcete, aby `Product` byl *smazán*, pokud se stane "osiřelým" (tj. bez `Category`)? Chcete-li zvolit toto chování, použijte možnost [orphanRemoval](#) uvnitř `Category`:

```

// src/Entity/Category.php

// ...

#[ORM\OneToMany(targetEntity: Product::class, mappedBy: 'category',
orphanRemoval: true)]
private array $products;

```

2.9 Kód z praktického příkladu

Nejprve vytvoříme entitu `Product` a pak entitu `Category`. Dále pomocí průvodce vytvoříme v entitě

`Product` další sloupec `category`, který bude typu `relation`. Zde nastavit v průvodci `ManyToOne`, bude mířit do `Category`.

2.9.1 Výpisy souborů

Category.php

```
<?php

namespace App\Entity;

use App\Repository\CategoryRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: CategoryRepository::class)]
class Category
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $name = null;

    #[ORM\OneToMany(mappedBy: 'category', targetEntity: Product::class)]
    private Collection $products;

    public function __construct()
    {
        $this->products = new ArrayCollection();
    }

    /**
     * @ORM\OneToMany(targetEntity="Product",mappedBy="category")
     *
     */

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getName(): ?string
    {

```

```

        return $this->name;
    }

    public function setName(string $name): static
    {
        $this->name = $name;

        return $this;
    }

    /**
     * @return Collection<int, Product>
     */
    public function getProducts(): Collection
    {
        return $this->products;
    }

    public function addProducts(Product $products): static
    {
        if (!$this->products->contains($products)) {
            $this->products->add($products);
            $products->setCategory($this);
        }

        return $this;
    }

    public function removeProducts(Product $products): static
    {
        if ($this->products->removeElement($products)) {
            // set the owning side to null (unless already changed)
            if ($products->getCategory() === $this) {
                $products->setCategory(null);
            }
        }

        return $this;
    }
}

```

Product.php

```
<?php

namespace App\Entity;

use App\Repository\ProductRepository;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: ProductRepository::class)]
class Product
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $name = null;

    #[ORM\Column(length: 255)]
    private ?string $quantity = null;

    #[ORM\ManyToOne(inversedBy: 'products')]
    #[ORM\JoinColumn(nullable: false)]
    private ?Category $category = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getName(): ?string
    {
        return $this->name;
    }

    public function setName(string $name): static
    {
        $this->name = $name;

        return $this;
    }
}
```

```

    public function getQuantity(): ?string
    {
        return $this->quantity;
    }

    public function setQuantity(string $quantity): static
    {
        $this->quantity = $quantity;

        return $this;
    }

    public function getCategory(): ?Category
    {
        return $this->category;
    }

    public function setCategory(?Category $category): static
    {
        $this->category = $category;

        return $this;
    }
}

```

ProductController.php

```

<?php

namespace App\Controller;

use App\Entity\Category;
use App\Entity\Product;
use App\Repository\CategoryRepository;
use App\Repository\ProductRepository;
use Doctrine\Persistence\ManagerRegistry;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Doctrine\ORM\EntityManagerInterface;

```

```

class ProductController extends AbstractController
{
    #[Route('/product', name: 'product')]
    public function index(EntityManagerInterface $entityManager): Response
    {
        $category = new Category();
        $category->setName('Computer Peripherals');

        $product = new Product();
        $product->setName('Keyboard');
        $product->setQuantity(50);

        // relates this product to the category
        $product->setCategory($category);

        $entityManager->persist($category);
        $entityManager->persist($product);
        $entityManager->flush();

        return new Response(
            'Saved new product with id: ' . $product->getId()
            . ' and new category with id: ' . $category->getId()
        );
    }
}

```

//Načítání souvisejících objektů

//Když potřebujete načíst související objekty, váš pracovní postup vypadá jako předtím. Nejprve načtěte \$productobjekt a poté získejte přístup k jeho souvisejícímu Categoryobjektu:

```

#[Route('/zobrazprodukty1', name: 'product1')]
public function showProductsByCategory1(ProductRepository
$productRepository)
{
    $id=1;
    $product = $productRepository->find($id);
    $categoryName=$product->getCategory()->getName();
    return new Response($categoryName );
}

```

//Protože jsme mapovali volitelnou OneToMany stranu, můžete se také dotazovat opačným směrem

```
#[Route('/zobrazprodukty2', name: 'product2')]
public function showProductsByCategory2(CategoryRepository
$categoryRepository)
{
    $categoryId=1;
    $category = $categoryRepository->find($categoryId);

    $products = $category->getProducts();
    dump(get_class($category));
    //$nazevproduktu=$products
    return new Response("pokus");
}

#[Route('/zobrazprodukty3', name: 'product')]
public function showProductsByCategory3(ProductRepository
$productRepository)
{
    //$categoryId=1;
    $product = $productRepository->findby(['category' => '1']);
    $category = $productRepository->find( '1');

    // $data = [];

    // foreach ($product as $p) {
    //     $data[] = [
    //         'id' => $p->getId(),
    //         'name' => $p->getName(),
    //         // Přidejte další atributy produktu, které chcete zobrazit
    //     ];
    // }

    $html = '<ul>';
    foreach ($product as $p) {
        $html .= '<li>' . $p->getName() . '</li>';
    }
    $html .= '</ul>';
    dump($html);

    $categoryName=$category->getCategory()->getName();
    return new Response($categoryName." <br> ".$html );
}
```

```
}
```

product.html.twig

```
<h1>{{ category.name }}</h1>
<ul>
    {% for product in products %}
        <li>{{ product.name }}</li>
    {% endfor %}
</ul>
```

2.9.2 Výpis kategorie a produktů

```
#[Route('/zobrazprodukty4', name: 'product4')]
public function showProductsByCategory4(EntityManagerInterface
$entityManager)
{
    $categories = $entityManager->getRepository(Category::class)->findAll();
    return $this->render('vse.html.twig', [
        'categories' => $categories
    ]);
}
```

Pak v vse.html.twig

```
{% for category in categories %}
    <div class="category">
        <h2>{{ category.name }}</h2>
        <ul>
            {% for product in category.products %}
                <li>{{ product.name }}</li>
            {% endfor %}
        </ul>
    </div>
{% endfor %}
```

3 Profiler

Nejprve zkusit instalaci takto

```
composer require --dev symfony/profiler-pack
```

Případně podle tohoto návodu

[Profiler: Your Debugging Best Friend > Harmonious Development with Symfony 6 | SymfonyCasts](#)

```
git add .
```

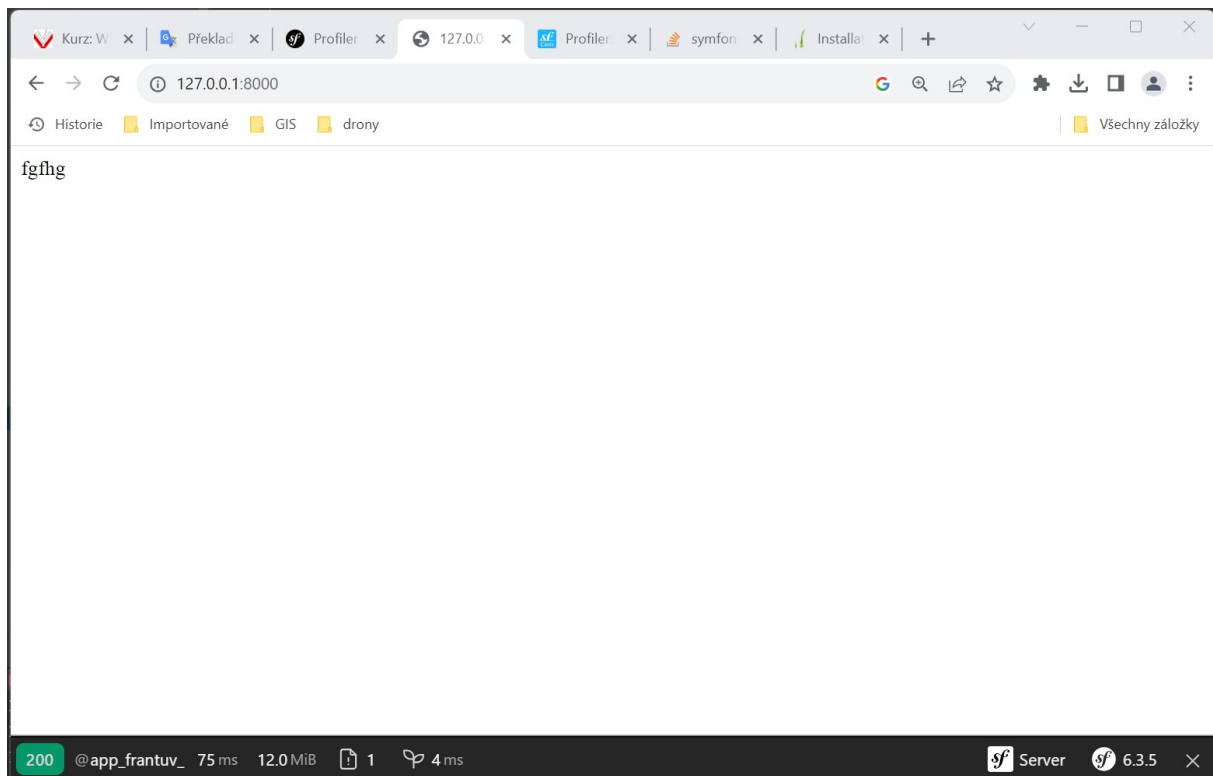
```
git commit -m "Added some Tiwgggy goodness"
```

```
composer require debug
```

```
git status
```

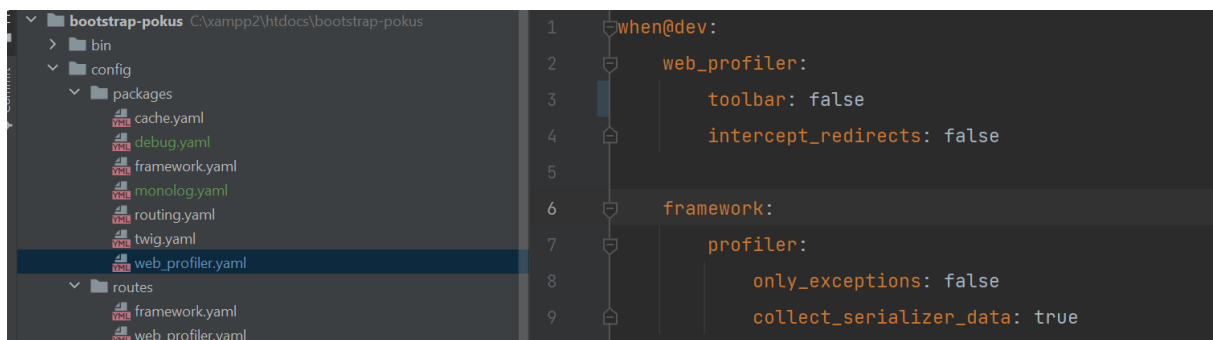
3.1 Pokud po spuštění projektu nejde profiler

Panel nástrojů je připojen těsně před uzavřením značky `<body>`. Proto se zobrazí pouze v případě, že váš dokument HTML obsahuje `a<body>`



3.2 Vypnutí profileru

v souboru config-web_profiler.yml toolbar nastavit na false.



4 Roletka ve formuláři

<https://symfony.com/doc/current/reference/forms/types/choice.html>

<https://symfony.com/doc/current/reference/forms/types/choice.html>

Každý Symfony formulář je spojen s nějakou entitou.

php bin/console make:form

```

/ create a task and give it some dummy data for this example
$task = new Task();
$task->setTask('Write a blog post');
$task->setDueDate(new \DateTime('tomorrow'));
$form = $this->createFormBuilder($task)
    ->add('task', TextType::class)
    ->add('dueDate', DateType::class)
    ->add('dueDate', DateType::class)
    ->add('souhlas', ChoiceType::class, [
        'choices' => [
            'Maybe' => null,
            'Yes' => true,
            'No' => false,
        ],
    ])
    ->add('save', SubmitType::class, array('label' => 'Create Task'))
    ->getForm();
return $this->render('default/new.html.twig', array(
    'form' => $form->createView(),
));
}
}

```

Přednáška

5 Form -Dropdownlist

5.1 Vyplnění dropdownlist formuláře z entity

Vytvoříme sklad

#	Název	Typ	Porovnávání	Vlastnosti	Nulový	Výchozí	Komentáře	Další	Operace
☐ 1	id	int(11)			Ne	Žádná		AUTO_INCREMENT	Změnit Odstranit Více
☐ 2	nazev	varchar(255)	utf8_unicode_ci		Ne	Žádná			Změnit Odstranit Více
☐ 3	cena	double			Ne	Žádná			Změnit Odstranit Více
☐ 4	mistnost	varchar(255)	utf8_unicode_ci		Ne	Žádná			Změnit Odstranit Více

Vytvoříme entitu Sklad

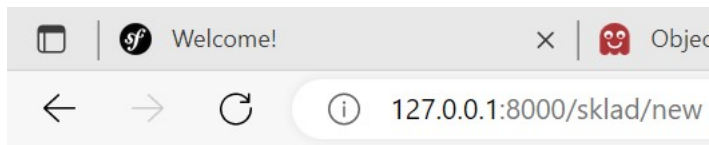
php bin/console doctrine:generate:entity

Dále updatujeme do databáze

```
php bin/console doctrine:schema:update --force
```

a vytvoříme CRUD

```
php bin/console generate:doctrine:crud
```



Sklad creation

Nazev

Cena

Místnost

- [Back to the list](#)

Nyní chceme, aby se místnost naplňovala tabulky Místnost

Vytvoříme tabulku Místnost

```
php bin/console doctrine:generate:entity
```

Dále updatujeme do databáze

```
php bin/console doctrine:schema:update --force
```

	#	Název	Typ	Porovnávání	Vlastnosti	Nulový	Výchozí	Komentáře	Další	Operace
☐	1	id	int(11)			Ne	Žádná		AUTO_INCREMENT	Změnit Odstranit Více
☐	2	popis	varchar(255)	utf8_unicode_ci		Ne	Žádná			Změnit Odstranit Více

☐ Zaškrtnout vše Zaškrtnuté: Projít Změnit Odstranit Primární Unikátní Index Prostorový Fulltext

Tabulku naplníme daty

← T →						id	popis
☐		Upravit		Kopírovat	 Odstranit	1	1.A
☐		Upravit		Kopírovat	 Odstranit	2	1.B

Vytvoření roletky ve formuláři a naplnění

<https://symfony.com/doc/current/reference/forms/types/entity.html>

Základní použití

Typ entity má pouze jednu požadovanou možnost: entitu, která by měla být uvedena v poli výběru:

```
use App\Entity\User;
use Symfony\Bridge\Doctrine\Form\Type\EntityType;
// ...
```

```
$builder->add('users', EntityType::class, [
    // looks for choices from this entity
    'class' => User::class,

    // uses the User.username property as the visible option string
    'choice_label' => 'username',

    // used to render a select box, check boxes or radios
    // 'multiple' => true,
    // 'expanded' => true,
]);
```

Tedy prakticky

```
<?php

namespace AppBundle\Form;

use Symfony\Component\Form\AbstractType;
```

```
use Symfony\Component\Form\FormBuilderInterface;

use Symfony\Component\OptionsResolver\OptionsResolver;

class SkladType extends AbstractType
{
    /**
     * {@inheritdoc}
     */

    public function buildForm(FormBuilderInterface $builder, array $options)
    {
        $builder->add('nazev')->add('cena')->add('mistnost');
    }

    /**
     * {@inheritdoc}
     */

    public function configureOptions(OptionsResolver $resolver)
    {
        $resolver->setDefaults(array(
            'data_class' => 'AppBundle\Entity\Sklad'
        ));
    }
}
```

```
/**
 * {@inheritdoc}
 */

public function getBlockPrefix()
{
    return 'appbundle_sklad';
}

}
```

Změníme na a nezapomene `use AppBundle\Entity\Mistnost;`

```
?php

//https://symfony.com/doc/current/reference/forms/types/entity.html

namespace AppBundle\Form;

use AppBundle\Entity\Mistnost;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;
use Symfony\Bridge\Doctrine\Form\Type\EntityType;
```

```
class SkladType extends AbstractType
{
    /**
     * {@inheritdoc}
     */

    public function buildForm(FormBuilderInterface $builder, array $options)
    {
        $builder->add('nazev')

            ->add('cena');

        $builder->add('mistnost', EntityType::class, [

            // looks for choices from this entity

            'class' => Mistnost::class,

            // uses the User.username property as the visible option string

            'choice_label' => 'popis',

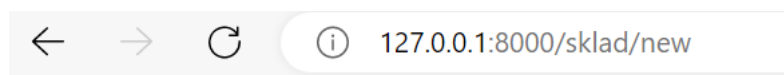
            'choice_value' => 'popis',

            // used to render a select box, check boxes or radios

            'multiple' => true,
```

```
// 'expanded' => true,  
  
});  
  
}/**
```

Po spuštění:



Sklad creation

Nazev
Cena
Mistnost

- [Back to the list](#)

Po odeslání a vyplnění new se objeví chybová hláška:

Symfony 3 Object of class could not be converted to string

Pro odstranění je nutné například vložit

Do entity Mistnost na konec vložíme

(<https://stackoverflow.com/questions/47479268/symfony-3-object-of-class-could-not-be-converted-to-string>):


```

public function __toString() {

    return $this->getPopis();

}

}

```

Vše pracuje OK:

Sklads list

Id	Nazev	Cena	Mistnost	Actions
<u>1</u>	jk	1	1.A	show edit
Create a new sklad				

5.2 Použití vlastního dotazu pro entity

Pokud chcete vytvořit vlastní dotaz pro použití při načítání entit (např. chcete vrátit pouze některé entity nebo je potřebujete objednat), použijte volbu [query builder](#) :

```

use App\Entity\User;
use Doctrine\ORM\EntityRepository;
use Symfony\Bridge\Doctrine\Form\Type\EntityType;
// ...

```

```

$builder->add('users', EntityType::class, [
    'class' => User::class,
    'query_builder' => function (EntityRepository $er) {
        return $er->createQueryBuilder('u')
            ->orderBy('u.username', 'ASC');
    },
    'choice_label' => 'username',
]);

```

Tedy prakticky, nezapomenout use Repository

```
use Doctrine\ORM\EntityRepository;
```

```
public function buildForm(FormBuilderInterface $builder, array $options)
{
    $builder->add('nazev')
        ->add('cena');

    $builder->add('mistnost', EntityType::class, [
        'class'=>Mistnost::class,
        'query_builder' => function (EntityRepository $er) {
            return $er->createQueryBuilder('m')
                ->orderBy('m.popis', 'DESC');
        },
        'choice_label' => 'popis',
    ]);
}
```

Pokud chceme vložit z tabulky Mistnost id do tabulky Sklad musíme:

Přetypovat v tabulce Sklad sloupec mistnost na int

☐	1	id 	int(11)
☐	2	nazev	varchar(255) utf8_unicode_ci
☐	3	cena	double
☐	4	mistnost	int(11)

Opravit

Entitu

```
/**
 * @var int
 *
 * @ORM\Column(name="mistnost", type="string", length=255)
 */
private $mistnost;
```

Pak opravit soubor s formulářem SkladType.php

```
public function buildForm(FormBuilderInterface $builder, array $options)
{
    $builder->add('nazev')
        ->add('cena');

    $builder->add('mistnost', EntityType::class, [
        // looks for choices from this entity
    ]
```

```
'class' => Mistnost::class,  
  
    // uses the User.username property as the visible option string  
  
    'choice_label' => 'id',  
  
    'choice_value' => 'id',  
  
    // used to render a select box, check boxes or radios  
  
    // 'multiple' => true,  
  
    // 'expanded' => true,  
  
]);
```

V Entitě Mistnost je stále funkce

```
public function __toString() {  
    return $this->getPopis();  
}
```

Sklad creation

Nazev

Cena

Mistnost ▼

- [Back to the list](#)

Případně takto, kdy uživatel vidí popis, ale do skladu se uloží id místnosti

```
$builder->add('mistnost', EntityType::class, [  
    // looks for choices from this entity  
    'class' => Mistnost::class,  
  
    // uses the User.username property as the visible option string  
    'choice_label' => 'popis',  
    'choice_value' => 'id',  
  
    // used to render a select box, check boxes or radios  
    'multiple' => true,  
    'expanded' => true,  
]);
```

Poznámka

toString

Funkce __toString() vrací obsah řetězce prvku.

Tato funkce vrací obsah řetězce, který je přímo v prvku – nikoli obsah řetězce, který je uvnitř potomků tohoto prvku!

<https://www.youtube.com/watch?v=dD4nsyZOE-Q>

Nebo příklad

```
<?php
```

```
// Create Your Class
```

```
class MyClass
```

```
{
```

```
    public $name;
```

```
    public function __construct($name)
```

```
    {
```

```
        $this->name = $name;
```

```
    }
```

```
    public function __toString()
```

```
    {
```

```
        try
```

```
        {
```

```
            return (string) $this->name;
```

```
        }
```

```
        catch (Exception $exception)
```

```
        {
```

```
            return " ";
```

```
        }
```

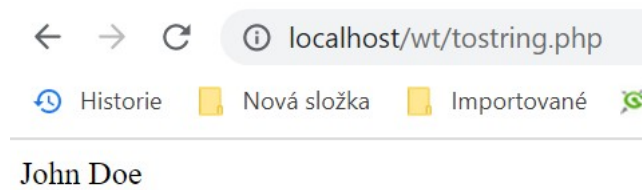
```
    }
```

```
}
```

```
$classObject = new MyClass('John Doe');
```

```
echo $classObject;
```

Výsledek:



Metoda `__toString()` je volána automaticky, když používáme funkci `echo` nebo `print` k vypsání proměnné, k níž byla přiřazena instance třídy. Příklad:

```
<?php

class UzitiToString{

    private $vlastnost;

    public function __construct($vlastnost){

        $this->vlastnost = $vlastnost;

    }

    public function __toString(){

        return $this->vlastnost;

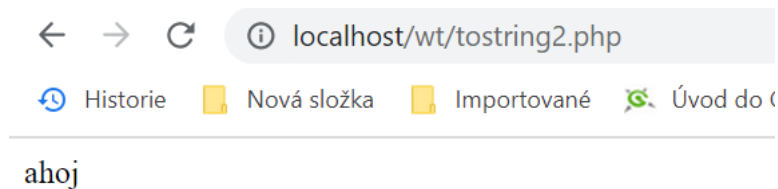
    }

}

$ins = new UzitiToString('ahoj');

echo $ins;

?>
```



6 Generované PDF z HTML v Symfony 6

Zdroj: <https://www.binaryboxtuts.com/php-tutorials/generating-pdf-from-html-in-symfony-6/>

Instalace pomocí composer:

```
composer create-project symfony/website-skeleton symfony-6-pdf
```

Instalace pomocí Symfony CLI:

```
symfony new symfony-6-pdf--full
```

Poté nainstalujeme tento balíček ***dompdf/dompdf***, tento balíček se použije při převodu HTML do PDF. Spust'te tento příkaz pro instalaci balíčku:

```
composer require dompdf/dompdf
```

Nyní vytvoříme náš řadič, spust'te tento příkaz pro vytvoření řadiče:

```
1 php bin/console make:controller PdfGenerator
```

Po vytvoření ovladače přidejte tyto kódy:

```
1 <?php
2
3 namespace App\Controller;
4
5 use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6 use Symfony\Component\HttpFoundation\Response;
7 use Symfony\Component\Routing\Annotation\Route;
8 use Dompdf\Dompdf;
9
```



```

class PdfGeneratorController extends AbstractController
{
    #[Route('/pdf/generator', name: 'app_pdf_generator')]
10    public function index(): Response
11    {
12        // return $this->render('pdf_generator/index.html.twig', [
13        //     'controller_name' => 'PdfGeneratorController',
14        // ]);
15        $data = [
16            'imageSrc' => $this->imageToBase64($this->getParameter('kernel.project_dir')
17            'name'      => 'John Doe',
18            'address'   => 'USA',
19            'mobileNumber' => '0000000000',
20            'email'     => 'john.doe@email.com'
21        ];
22        $html = $this->renderView('pdf_generator/index.html.twig', $data);
23        $dompdf = new Dompdf();
24        $dompdf->loadHtml($html);
25        $dompdf->render();
26
27        return new Response (
28            $dompdf->stream('resume', ["Attachment" => false]),
29            Response::HTTP_OK,
30            ['Content-Type' => 'application/pdf']
31        );
32    }
33
34    private function imageToBase64($path) {
35        $path = $path;
36        $type = pathinfo($path, PATHINFO_EXTENSION);
37        $data = file_get_contents($path);
38        $base64 = 'data:image/' . $type . ';base64,' . base64_encode($data);
        return $base64;
    }
}

```

Nyní aktualizujeme soubor pohledu, který byl vytvořen při vytváření ovladače. Otevřete soubor ***/templates/pdf_generator/index.html.twig*** a odstraňte stávající kódy a změňte jej na tyto:

/templates/pdf_generator/index.html.twig

```

1    <!DOCTYPE html>
2    <html>

```

```

    <head>
      <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
      <title>Resume</title>
3    </head>
4    <body>
5      <div style="margin: 0 auto;display: block;width: 500px;">
6        <table width="100%" border="1">
7          <tr>
8            <td colspan="2">
9              
10             </td>
11          </tr>
12          <tr>
13            <td>Name:</td>
14            <td>{{name}}</td>
15          </tr>
16          <tr>
17            <td>Address:</td>
18            <td>{{address}}</td>
19          </tr>
20          <tr>
21            <td>Mobile Number:</td>
22            <td>{{mobileNumber}}</td>
23          </tr>
24          <tr>
25            <td>Email:</td>
26            <td>{{email}}</td>
27          </tr>
28        </table>
      </div>
    </body>
  </html>

```

Než přistoupíte k dalšímu kroku, ujistěte se, že máte soubor obrázku v adresáři /public, protože obrázek přidáme při generování souboru PDF. Zde je můj vzorový obrázek, který je uložen na ***/public/img/profile.png***

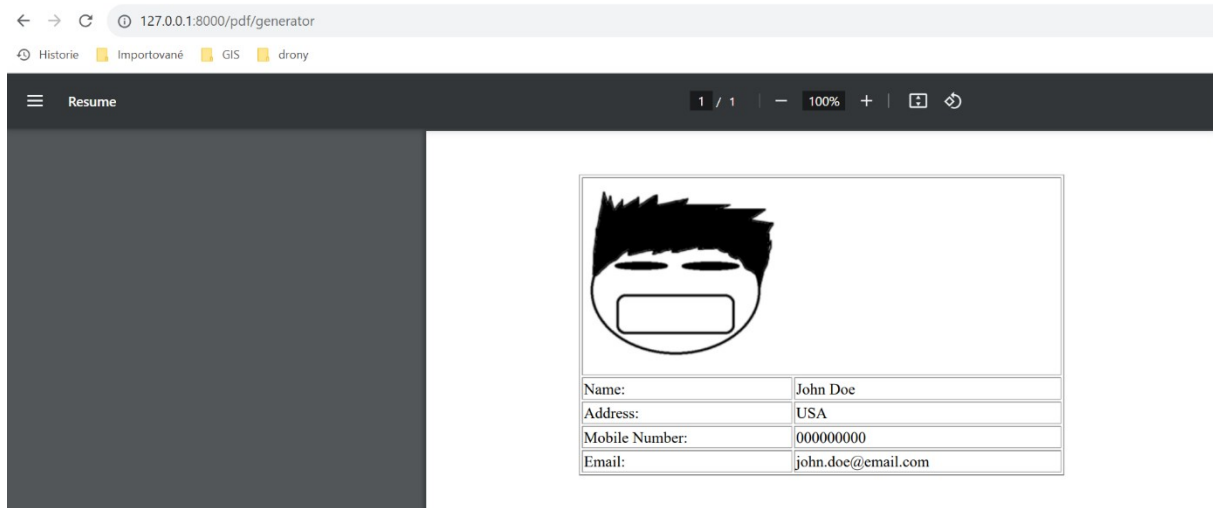
Odkomentujeme knihovnu gd v souboru php.ini

extension=gd

symfony server:start

Po úspěšném spuštění aplikace otevřete v prohlížeči tuto adresu URL:

1 <https://localhost:8000/pdf/generator>



7 Data z html formuláře do routy (get)

```
{% block body %}
```

```
{% if error %}
```

```
<div>{{ error.messageKey|trans(error.messageData, 'security') }}</div>
```

```
{% endif %}
```

```
<form action="{{ path('zobraz1') }}" method="post">
```

```
<label for="id">Zadej číslo kategorie</label>
```

```
<input type="text" id="id" name="id" ">
```

```
<button type="submit">login</button>
```

```
</form>
```

```
{% endblock %}
```

Pak v controlleru

```
#[Route('/zobrazprodukty2', name: 'product2', Request $request
)]
public function showProductsByCategory2(CategoryRepository
$categoryRepository)
{
    $categoryId = $request->query->get('id');
    //$categoryId=1;
    $category = $categoryRepository->find($categoryId);

    $products = $category->getProducts();
    dump(get_class($category));
    //$nazevproduktu=$products
    return new Response("pokus");
}
```